Hands-on 2: Lexical Analysis with FSA

Ruiz Verduzco Edson Omar 219539237

ICOM
Seguridad en la Información

Prof. JOSÉ ANTONIO AVIÑA MÉNDEZ
2026A

1. Introducción y Justificación del Proyecto

INTRODUCCIÓN:

El presente documento detalla la integración de las prácticas *Hands-on 1* y *Hands-on 2* correspondientes a la materia de Compiladores. En lugar de abordar ambas tareas como ejercicios aislados, se ha optado por una construcción progresiva y unificada, creando una solución mucho más cercana al funcionamiento real de un sistema de compilación.

En la primera etapa (*Hands-on 1*), se desarrolló la arquitectura base de una Máquina Virtual (VM) orientada a objetos en C++, simulando el hardware esencial (memoria, registros, unidad de control) y el ciclo máquina fundamental para ejecutar un programa.

En la segunda etapa (*Hands-on 2*), se implementó un Autómata Finito de Estados (FSA) basado en diagramas de transiciones, cuyo propósito es el análisis léxico para la validación de cifras numéricas (números sin signo, decimales y notación científica).

La relación entre ambas es intrínseca: la Unidad de Control de la VM requiere decodificar instrucciones mediante un Analizador Léxico (*Lexer*). Al integrar el FSA dentro de la etapa de decodificación de la VM, el sistema adquiere la capacidad de validar tokens numéricos rigurosamente antes de proceder a la ejecución, logrando un diseño robusto, ordenado y alineado con los requerimientos académicos.

Objetivo General

Desarrollar una Máquina Virtual orientada a objetos en C++ con la capacidad de cargar, decodificar y ejecutar instrucciones de ensamblador simulado, integrando un módulo de análisis léxico (basado en un autómata finito) para la validación estricta de tokens numéricos antes de su procesamiento.

Objetivos Específicos

- **Construcción Arquitectónica:** Implementar la estructura base de la VM, incluyendo memoria principal, CPU, registros de propósito general y específico (AL, AH, BL, BH, PC, IR, ACC, MAR, MBR), y la Unidad de Control.
- **Definición de Instrucciones:** Establecer un conjunto reducido de operaciones válidas (`START`, `MOV`, `ADD`, `STO`, `END`).
- **Simulación del Ciclo Máquina:** Orquestar correctamente las etapas de *Fetch* (búsqueda), *Decode* (decodificación) y *Execute* (ejecución), monitoreando el estado de los registros.
- **Análisis Léxico:** Desarrollar un Autómata Finito de Estados capaz de aceptar cadenas numéricas válidas y rechazar sintaxis incorrectas.
- **Integración Funcional:** Acoplar el FSA en la fase de *Decode* de la Unidad de Control para garantizar que la VM solo procese instrucciones sintáctica y léxicamente válidas.

Arquitectura y Módulos del Sistema

Módulo / Clase	Responsabilidad Principal
Memory	Almacenamiento del programa y los datos en tiempo de ejecución.
Registers	Representación del banco de registros (generales y de control como PC e IR).
Instruction	Modelado de una instrucción individual (nombre, argumentos, longitud).
ProgramLoader	Encargado de leer el código fuente y cargarlo en la memoria principal.
CPU	Simulación del procesador; expone las operaciones matemáticas y lógicas.
ControlUnit	Orquestador del sistema. Coordina el ciclo *Fetch - Decode - Execute*.
Lexer	Separación de las líneas de código fuente en *tokens* individuales.
NumberFSA	Autómata finito del *Hands-on 2*. Valida estrictamente la estructura de tokens numéricos.

Conjunto de Instrucciones (Instruction Set)

Instrucción	Sintaxis	Descripción
Inicio	START	Define el punto de entrada del programa.
Transferencia	MOV reg, valor	Asigna un valor numérico o el contenido de otro registro a un registro destino.
Aritmética	ADD reg1, reg2	Suma el contenido de dos registros y almacena el resultado.
Almacenamiento	STO addr	Guarda el valor de un registro específico en una dirección de memoria.
Paro	END / STOP	Finaliza la ejecución del programa y detiene la Máquina Virtual.

Flujo general de ejecución

El proceso integrado opera de la siguiente manera:

1. El *ProgramLoader* copia las instrucciones hacia memoria.
2. La *ControlUnit* avanza a través del programa mediante el contador de programa (*PC*).
3. Durante *Fetch*, se obtiene de memoria la instrucción señalada.
4. En *Decode*, el *Lexer* separa la instrucción en tokens, clasificándolos según su tipo.
5. Si se detecta un token numérico, se envía al *NumberFSA* para validar su correcto formato.
6. Si todos los tokens son válidos, en *Execute* la VM realiza la operación indicada, ya sea mover datos, sumar registros o almacenar resultados.

Este flujo añade una capa de verificación previa, fortaleciendo el sistema y acercándose al pipeline real de un compilador.

Estructura de Archivos del Proyecto

```
HandsOn2_FSA/
│
├── src/
│   ├── main.cpp
│   ├── NumberFSA.h
│   └── NumberFSA.cpp
│
└── docs/
    ├── diagrama_fsa.png
    └── evidencia.md (o PDF)
```

Diagrama Oficial:

Válido:

25

12.5

3.2E4

3.2E+4

3.2E-4

0.34

No Válido:

E4

7E

.5 (normalmente NO se acepta según el diagrama del profe)

5. (tampoco)

3E+ (incompleto)

Estados formales

Estado	Significado

q0 Inicio, aún no se ha leído nada

q1 Parte entera (al menos un dígito)

q2 Se detectó punto decimal

q3 Parte decimal (al menos un dígito)

q4 Se detectó 'E'

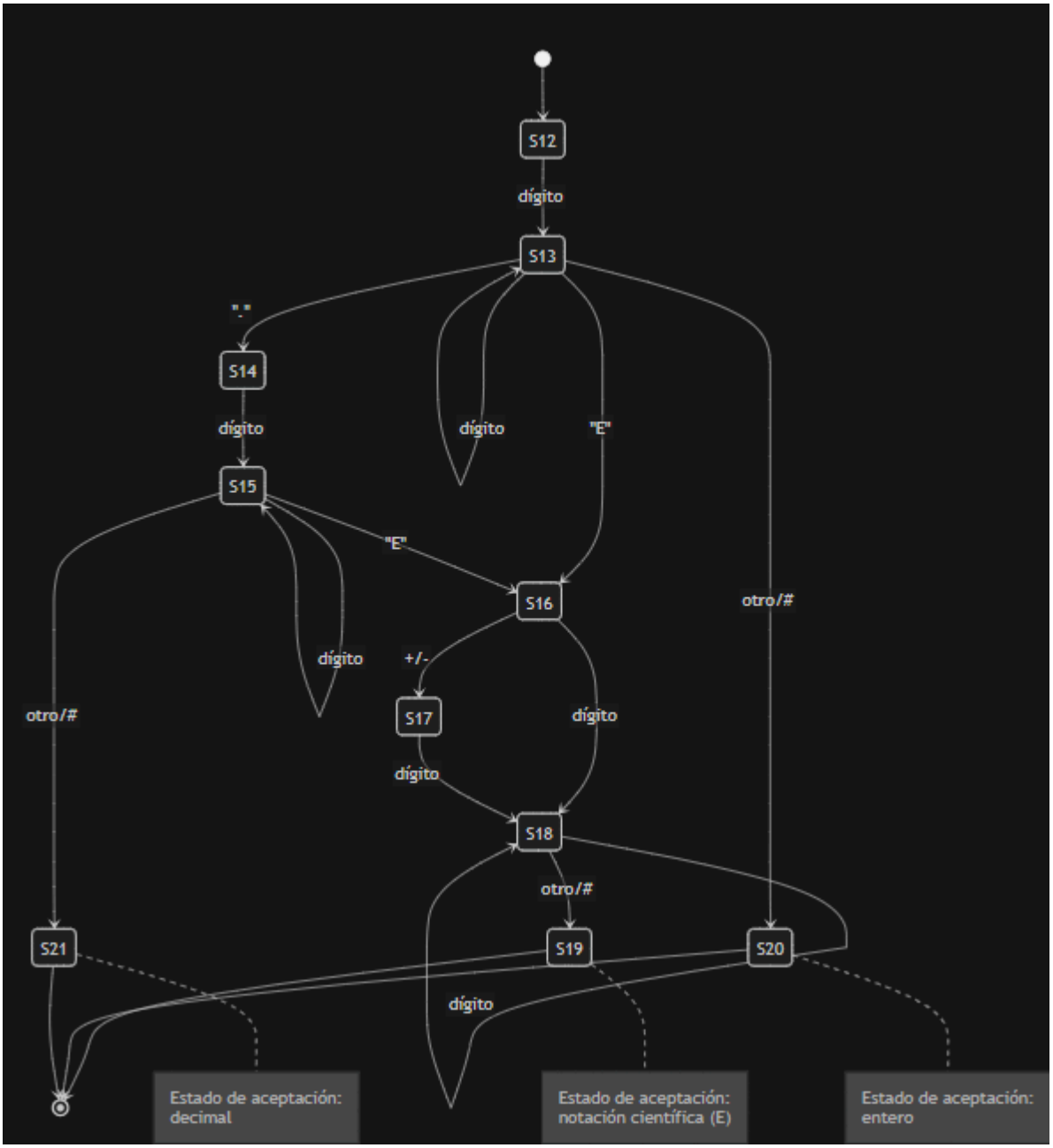
q5 Signo después de E (+ o -)

q6 Parte del exponente (al menos un dígito) – **aceptación**

TABLA DE TRANSICIONES

Estado	D (dígito)	.	E	+ / -	Otro / #	Tipo
12	13	—	—	—	**rechaza**	Inicio
13	13	14	16	—	20	Entero
14	15	—	—	—	**rechaza**	Punto sin dígito aún
15	15	—	16	—	21	Decimal

16	—	—	—	17	**rechaza**	E, esperando signo o dígito
17	18	—	—	—	**rechaza**	Signo después de E
18	18	—	—	—	19	Exponente
19	—	—	—	—	ACEPTA	Aceptación
20	—	—	—	—	ACEPTA	Aceptación
21	—	—	—	—	ACEPTA	Aceptación



Conclusión

La integración entre *Hands-on 1* y *Hands-on 2* da lugar a un proyecto más sólido y coherente. La Máquina Virtual constituye la base estructural del sistema, mientras que el autómata provee un componente crítico del análisis léxico dentro de la decodificación de instrucciones. Esta unión no solo fortalece la arquitectura general, sino que además cumple fielmente con los requisitos de la materia, donde se solicita la validación de tokens mediante autómatas finitos.